

```
<div style="background:#000;padding:24px;text-align:center;">

</div>

<style>
body { font-family: Arial, Helvetica, sans-serif; }
h1, h2 { color: #ffcc00; }
.section-header { background:#000; color:#ffcc00; padding:8px 12px; }
.muted { color:#666; }
</style>
```

CONSOLIDATED DOCUMENTATION — Projeto BRF

1) Overview

- Projeto BRF: testes E2E com Cypress para integração com o sistema Velox.
- Estrutura principal: `cypress/e2e/`, `cypress/fixtures/`, `cypress/support/`, `cypress/support/utils/`.

2) Quick Start

- Requisitos: Node.js v14+ (v16+ recomendado), npm/yam.
- Instalação:

```
npm install
```

- Abnir interface interativa:

```
npm run cy:open
```

- Rodar em modo headless:

```
npm test
```

3) Configuração

- `cypress.config.js` define `baseUrl` (`https://velox2.velox-by-invent.com`), `timeouts`, gravação de vídeo, `chromeWebSecurity` false e flags para evitar detecção de automação no Chromium.
- `process.env.NODE_TLS_REJECT_UNAUTHORIZED = '0'` está definido para ambientes com certificados autoassinados.

4) Test suites

- Suites organizadas por CT dentro de `cypress/e2e/`. Exemplos: `CT002_Regressivo_1_Etapa`, `CT004_OT_Pendente`, `CT017_Tord_Nao_Confirma`, `CT018_Regressivo_Tord_Confirma`, `CT027_Confirma_Estoque`, `CT031_Sequencia_Palete`.
- Padrões de spec: uso de fixtures, `before()`/`beforeEach()`, iterações dinâmicas via `forEach`, `intercepts`, e `cy.wait()` quando necessário.

5) Fixtures & Data Generation

- `cypress/fixtures/tordDados.json` — amazena dados gerados para TORD (ex.: `DOCNUM`, `TANUM`, `LENUM`, `ZVFDAT`, `CHARG`, `CREDAT`, `CRETIM`).
- `cypress/support/utils/gerarDados.js` — helpers para gerar números e formatos específicos, e funções `gerarDadosTORD()` / `carregarDadosTORD()` que escrevem/leem `tordDados.json` e populam `Cypress.env`.

6) CI (GitHub Actions)

- Foi adicionado um workflow em `.github/workflows/ci.yml` que:
 - Roda em push e pull_request nas branches main/master;
 - Instala dependências com `npm ci` e executa `npx cypress run`;
 - Faz cache do `~/.npm` e `~/.cache/Cypress` para acelerar runs;
 - Faz upload de `cypress/videos` e `cypress/screenshots` como artifacts.

7) Troubleshooting

- Erros TLS: `NODE_TLS_REJECT_UNAUTHORIZED = '0'` pode ajudar em ambientes de dev/CI com certificados inválidos.
- Timeouts: ajustar `pageLoadTimeout` e `defaultCommandTimeout` em `cypress.config.js`.
- Falhas intermitentes no UI: usar `cy.on('uncaught:exception', () => false)` com cautela para evitar mascarar problemas reais.

8) Next steps recomendados

- Configurar secrets no repositório (ex.: `CYPRESS_BASE_URL`, `CYPRESS_RECORD_KEY` se for usar gravação no Dashboard).
- Adicionar um workflow condicional para execução em matrizes (browsers/versions) se necessário.
- Gerar um PDF consolidado para stakeholders (opcional).

Testes — descrição por suite e spec

<!-- incluído de tests_descriptions.md -->

CT002_Regressivo_1_Etapa

- `01_TORD_Pendente.cy.js`: Gera dados únicos de `gerarDadosTORD()` e envia um XML TORD com status `Pendente` para o endpoint `mbrf-tord-v1`. Valida que a requisição foi aceita (200/201/202) e que o corpo de resposta não está vazio.
- `02_Reab_Partel.cy.js`: (fluxo) Inicia reabastecimento parcialmente via XML/integração — prepara estado para confirmações posteriores.
- `03_TOCO_Confirma.cy.js`: Envia XML TOCO para confirmar OT criada pelo TORD; valida aceitação do TOCO.
- `04_Confirma_Reab.cy.js`: Valida pela UI/integracao finalização de reabastecimento (confirmações internas).

CT004_OT_Pendente

- `01_TORD_Pendente.cy.js`: Idêntico ao padrão de TORD - envia XML TORD `Pendente`, valida resposta.
- `02_Verifica_OT.cy.js`: Valida via UI/endpoint que a OT existe e está em estado esperado (`pendente` ou `criado`).
- `03_Finaliza_OT.cy.js`: Finaliza a OT via TOCO/XML e valida processamento.

CT017_Tord_Nao_Confirma

- `01_TORD_Not_Zconf.cy.js`: Envia TORD com campo `ZCONF` vazio (não confirmado) e valida aceitação do XML. Usado para testar fluxo onde não há confirmação automática.
- `02_Verifica_Pendente.cy.js`: Verifica se a OT ficou com status `pendente` no sistema.
- `03_Finaliza_OT.cy.js`: Executa a finalização manual/forçada da OT (fluxo de correção).

CT018_Regressivo_Tord_Confirma

- `01_TORD_Confirma.cy.js`: Envia TORD com `ZCONF` marcado para simular TORD já confirmado no envio; valida resposta do endpoint.
- `02_Confirma_Integra_Estoque.cy.js`: Garante integração que atualiza o estoque/posições a partir do TORD confirmado.

CT027_Confirma_Estoque

- 01_TORD_Pendente.cy.js: Envia TORD Pendente — mesmo padrão de geração de dados.
- 02_Reab_Partel.cy.js: Cria reabastecimento parcial via integração.
- 03_TOCO_Confirma.cy.js: Confirma via TOCO/XML a OT criada.
- 04_Confirma_Reab.cy.js: Valida através do sistema (UI/integracao) que o reabastecimento foi confirmado e que o saldo/paleta foi atualizado.

CT031_Sequencia_Paleta

- 00_Sequencia_TORD_TOCO.cy.js: Gera 3 iterações completas (TORD + TOCO); para cada iteração insere TORD, aguarda processamento e faz TOCO, armazenando resultados em cypress/fixtures/tordSequencia_detailed.json. Testa robustez do processamento em sequência e retry logic do TOCO.
- 03_Confirma_Reab.cy.js: Usa a fixture gerada (tordSequencia_detailed.json) para, via UI, validar a existência da OT (TANUM), checar status Finalizada e confirmar que o LENUM (paleta) aparece corretamente no módulo de Saldo.

CT032_Cancela_Remessa

- 01_Integra_Remessa.cy.js: Integra remessa (envio inicial) para o fluxo de remessa.
- 02_Cancela_VerificaOnda.cy.js: Cancela a remessa criada e valida efeito sobre ondas/ondas de picking.

Integracao_Tord_Toco

- 1_TORD_Pendente.cy.js: Envia TORD Pendente (integração direta).
- 2_TOCO_Confirma.cy.js: Em sequência, envia TOCO para confirmar TORD; usado para testes de integração ponta-a-ponta sem UI.

Login

- Login.cy.js: Teste básico de login que preenche credenciais e aguarda a resposta da requisição de login.
- Reab1Etapa_Partel.cy.js / Reab1Etapa_Parte2.cy.js: Tests de fluxo que começam com login e executam ações no workspace relacionadas ao reabastecimento (parte 1 e 2).

Observações gerais sobre os testes

- Muitos specs dependem de integrações HTTP XML (mbrf-tord-v1, mbrf-zwmtoco-v1) e exigem que o ambiente baseUrl esteja acessível.
- Dados únicos gerados por cypress/support/utils/gerarDados.js são salvos em cypress/fixtures/tordDados.json para reuso entre specs.
- Specs de UI usam cy.intercept() e cy.wait() para sincronizar com o backend; alguns testes usam cy.on('uncaught:exception', () => false) para ignorar erros JS não relacionados.

Stakeholder Brief — Resumo Executivo

TL;DR

- Objetivo: validar fluxos críticos do sistema Velox via testes E2E automatizados com Cypress.
- Cobertura: suites organizadas por fluxo (ex.: CT002, CT004, CT017, CT018, CT027, CT031).
- Frequência recomendada: execução automática em PRs + agendamento diário para regressão completa.

Resumo executivo

- Testes automatizados verificam integração de transporte (TORD/OT), sequências de paleta e confirmações de estoque.
- Resultados: artefatos gerados em cada execução (vídeos e screenshots) e possibilidade de gravação no Cypress Dashboard.

Como interpretar os resultados

- Pass: fluxo validado com sucesso.
- Fail: falha pode indicar regressão no backend, instabilidade de ambiente ou mudanças na UI — revisar vídeo/screenshot e logs.
- Artefatos disponíveis: vídeos (cypress/videos/), screenshots (cypress/screenshots/) e logs de CI.

Ações recomendadas em caso de falha

1. Reproduzir localmente usando o mesmo spec falho com npx cypress open.
2. Verificar vídeo e screenshot no CI (artifacts) para entender o ponto de falha.
3. Validar se houve alteração recente no backend (endpoints) ou nos seletores da UI.

Entrega e distribuição

- O PDF consolidado é gerado pelo workflow e está disponível como artifact na execução das Actions.